

Lab Task 1: Student Records Management

Scenario:

You are building a **student record management system** for a university. Student records are stored by **Roll Number** and must always remain efficiently searchable.

To ensure consistent performance even as data grows, the system uses an **AVL Tree** instead of a plain BST to keep the records balanced after every insertion or deletion.

Data Model:

Each node represents a student record with:

- `int rollNo` (unique key)
- `string name`
- `float cgpa`

Core Functionalities:

- `addStudent(int rollNo, string name, float cgpa)`
- `findStudent(int rollNo)`
- `removeStudent(int rollNo)`
- `displayInorder()`

Advanced Operations:

- `getTopStudents(float threshold)` - returns all students with CGPA $\geq$ threshold
- `getTotalStudents()`

- `getHeight()`
- `isBalanced()` - verifies AVL property for all nodes

Test Cases

```
#include "gtest/gtest.h"

#include "StudentAVL.h"

// 1. Duplicate Roll Numbers Should Be Rejected

TEST(StudentAVLTest, RejectDuplicateRollNumbers) {

    StudentAVL avl;

    EXPECT_TRUE(avl.addStudent(101, "Alice", 3.5));

    EXPECT_FALSE(avl.addStudent(101, "Bob", 3.8)); // Duplicate roll
number

    EXPECT_EQ(avl.getTotalStudents(), 1);

}

// 2. Search for Non-Existing Roll Number Returns Empty Record

TEST(StudentAVLTest, FindNonExistingStudent) {

    StudentAVL avl;

    avl.addStudent(1, "A", 3.1);

    Student s = avl.findStudent(999);

    EXPECT_EQ(s.rollNo, -1);

    EXPECT_STREQ(s.name, "");

}
```

```
    EXPECT_FLOAT_EQ(s.cgpa, 0.0);
}

// 3. Left-Right Rotation Triggered
TEST(StudentAVLTest, LeftRightRotation) {
    StudentAVL avl;

    avl.addStudent(30, "A", 3.0);
    avl.addStudent(10, "B", 2.8);
    avl.addStudent(20, "C", 3.6); // triggers LR rotation

    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 3);
}

// 4. Right-Left Rotation Triggered
TEST(StudentAVLTest, RightLeftRotation) {
    StudentAVL avl;

    avl.addStudent(10, "A", 3.0);
    avl.addStudent(30, "B", 3.2);
    avl.addStudent(20, "C", 3.6); // triggers RL rotation

    EXPECT_TRUE(avl.isBalanced());
    EXPECT_LT(avl.getHeight(), 3);
}
```

```
// 5. Deleting Root Node Rebalances Correctly

TEST(StudentAVLTest, DeleteRootNode) {

    StudentAVL avl;

    avl.addStudent(50, "X", 3.2);

    avl.addStudent(30, "Y", 3.4);

    avl.addStudent(70, "Z", 3.6);

    EXPECT_TRUE(avl.removeStudent(50)); // Delete root

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_EQ(avl.getTotalStudents(), 2);

}

// 6. Query Students Within Roll Number Range

TEST(StudentAVLTest, RangeQueryByRollNumber) {

    StudentAVL avl;

    avl.addStudent(10, "A", 2.5);

    avl.addStudent(20, "B", 3.5);

    avl.addStudent(30, "C", 3.0);

    avl.addStudent(40, "D", 3.8);

    int count = 0;

    Student* subset = avl.getRangeOfStudents(15, 35, count);

    EXPECT_EQ(count, 2);

    EXPECT_EQ(subset[0].rollNo, 20);

}
```

```

        EXPECT_EQ(subset[1].rollNo, 30);

        delete[] subset;
    }

// 7. All Students with CGPA Above Threshold Returned
TEST(StudentAVLTest, HighAchieversQuery) {

    StudentAVL avl;

    avl.addStudent(1, "A", 3.1);

    avl.addStudent(2, "B", 3.9);

    avl.addStudent(3, "C", 2.7);

    avl.addStudent(4, "D", 3.8);

    int count = 0;

    Student* top = avl.getTopStudents(3.6, count);

    EXPECT_EQ(count, 2); // B and D

    delete[] top;
}

// 8. Tree Height Reflects Balance After Bulk Inserts
TEST(StudentAVLTest, HeightAfterMultipleInsertions) {

    StudentAVL avl;

    for (int i = 1; i <= 20; ++i)

        avl.addStudent(i, "X", 3.0f);

```

```

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_LT(avl.getHeight(), 8);
}

// 9. Delete Non-Existing Student Should Fail Gracefully
TEST(StudentAVLTest, DeleteNonExistingStudent) {

    StudentAVL avl;

    avl.addStudent(10, "A", 3.0);

    avl.addStudent(20, "B", 3.2);

    EXPECT_FALSE(avl.removeStudent(999)); // Non-existing roll number

    EXPECT_EQ(avl.getTotalStudents(), 2);

    EXPECT_TRUE(avl.isBalanced());
}

// 10. Large Sequential Inserts and Deletes Preserve Balance
TEST(StudentAVLTest, LargeInsertDeleteSequence) {

    StudentAVL avl;

    const int N = 100;

    for (int i = 1; i <= N; ++i)

        avl.addStudent(i, "S", 3.0f + (i % 10) * 0.1f);

    for (int i = 1; i <= N; i += 2)

        avl.removeStudent(i);

    EXPECT_TRUE(avl.isBalanced());
}

```

```

    EXPECT_EQ(avl.getTotalStudents(), N / 2);
}

// 11. Verify Re-Insertion After Deletion Maintains Balance
TEST(StudentAVLTest, ReinsertAfterDelete) {
    StudentAVL avl;

    avl.addStudent(10, "A", 3.3);

    avl.addStudent(20, "B", 3.4);

    avl.addStudent(30, "C", 3.5);

    avl.removeStudent(20);

    EXPECT_TRUE(avl.isBalanced());

    avl.addStudent(20, "B2", 3.9);

    EXPECT_TRUE(avl.isBalanced());

    Student s = avl.findStudent(20);

    EXPECT_FLOAT_EQ(s.cgpa, 3.9);
}

// 12. Boundary Roll Numbers (Minimum and Maximum)
TEST(StudentAVLTest, BoundaryRollNumbers) {
    StudentAVL avl;

    avl.addStudent(1, "Lowest", 3.1);

    avl.addStudent(9999, "Highest", 3.8);

    EXPECT_TRUE(avl.isBalanced());
}

```

```
Student s1 = avl.findStudent(1);

Student s2 = avl.findStudent(9999);

EXPECT_EQ(s1.rollNo, 1);

EXPECT_EQ(s2.rollNo, 9999);

}

// 13. Check Balance After Sequential Descending Inserts

TEST(StudentAVLTest, DescendingInsertionsBalance) {

    StudentAVL avl;

    for (int i = 10; i >= 1; --i)

        avl.addStudent(i, "X", 3.0);

    EXPECT_TRUE(avl.isBalanced());

    EXPECT_LT(avl.getHeight(), 6);

}

// 14. Delete All Students One by One

TEST(StudentAVLTest, DeleteAllStudents) {

    StudentAVL avl;

    avl.addStudent(10, "A", 3.1);

    avl.addStudent(20, "B", 3.2);

    avl.addStudent(30, "C", 3.3);
```



```

        avl.removeStudent(10);

        avl.removeStudent(20);

        avl.removeStudent(30);

        EXPECT_EQ(avl.getTotalStudents(), 0);

        EXPECT_TRUE(avl.isBalanced());
    }

// 15. Verify Height and Count Are Updated Correctly
TEST(StudentAVLTest, CountAndHeightAfterOperations) {

    StudentAVL avl;

    avl.addStudent(1, "A", 3.5);

    avl.addStudent(2, "B", 3.6);

    avl.addStudent(3, "C", 3.7);

    EXPECT_EQ(avl.getTotalStudents(), 3);

    avl.removeStudent(2);

    EXPECT_EQ(avl.getTotalStudents(), 2);

    EXPECT_TRUE(avl.isBalanced());
}

// 16. Update (Replace) Student's CGPA by Removing & Reinserting
TEST(StudentAVLTest, UpdateStudentRecord) {

    StudentAVL avl;

```

```

    avl.addStudent(55, "Liam", 2.9);

    avl.removeStudent(55);

    avl.addStudent(55, "Liam", 3.6);

    Student s = avl.findStudent(55);

    EXPECT_FLOAT_EQ(s.cgpa, 3.6);

    EXPECT_TRUE(avl.isBalanced());
}

// 17. Minimum and Maximum Roll Retrieval via Traversal
TEST(StudentAVLTest, MinMaxRollTraversal) {

    StudentAVL avl;

    avl.addStudent(100, "A", 3.1);

    avl.addStudent(50, "B", 3.3);

    avl.addStudent(150, "C", 3.2);

    Student min = avl.findStudent(50);

    Student max = avl.findStudent(150);

    EXPECT_EQ(min.rollNo, 50);

    EXPECT_EQ(max.rollNo, 150);
}

// 18. CGPA Threshold Edge Case (Exactly Equal to Threshold)
TEST(StudentAVLTest, ThresholdEqualToCGPA) {

```

```

StudentAVL avl;

avl.addStudent(10, "A", 3.6);

avl.addStudent(20, "B", 3.6);

avl.addStudent(30, "C", 3.5);

int count = 0;

Student* arr = avl.getTopStudents(3.6, count);

EXPECT_EQ(count, 2);

delete[] arr;

}

```

Lab Task 2: Dynamic Median Tracker Using AVL Tree

You are designing a **real-time median tracking system** for a financial analytics platform.

The system continuously receives **streaming numbers** — for example, stock prices, sensor readings, or incoming data packets.

After each insertion or deletion, the system must be able to **instantly report the current median** of all values seen so far.

A simple array or list would make median finding too slow ($O(n \log n)$ after sorting).

Instead, your task is to implement this using an **AVL Tree**, which maintains balance and enables efficient **rank-based lookups**.

The AVL must be *augmented* so that each node also stores the **size of its subtree** — allowing the median to be located in $O(\log n)$ time.

Data Model

Each node represents one numeric data point.

```
struct Node {
```

```
float value;    // The data value (e.g., 55.3)
int height;    // For AVL balancing
int subtreeSize; // Count of all nodes in this subtree (including itself)
Node* left;
Node* right;
};
```

Note:

Every rotation and insertion must also update `subtreeSize`, so it always reflects the total number of descendants under a node.

That way, you can navigate by **rank** to find the median position.

Functional Requirements

1. `insertValue(float val)`

- Inserts a new value into the AVL tree.
- Balances the tree after insertion.
- Updates both `height` and `subtreeSize` on every return step.
- Returns `true` on success, `false` if value already exists.

2. `deleteValue(float val)`

- Removes a node with the given value.
- If not found, returns `false`.
- Rebalances and updates `subtreeSize` afterward.

3. `getMedian()`

- Returns the **current median value** in $O(\log n)$ time.
- For odd number of nodes: middle element.
- For even number of nodes: average of the two middle elements.

- Uses a **k-th smallest element** helper based on subtree sizes:

4. `getCount()`

- Returns total number of values (root's `subtreeSize`).

5. `isBalanced()`

- Verifies AVL property ($|\text{height}(\text{left}) - \text{height}(\text{right})| \leq 1$ for all nodes).

6. `displayInorder()`

- Prints all elements in ascending order (for debugging).

Median Calculation

Let n = number of values in tree.

- If n is odd $\rightarrow$ median = value at position $(n + 1) / 2$
- If n is even $\rightarrow$ median = average of values at positions $n / 2$ and $(n / 2 + 1)$

```
#include "gtest/gtest.h"
#include "MedianAVL.h"

// 1. Simple Median (Odd Count)
TEST(MedianAVLTest, SimpleOddMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(30);
    avl.insertValue(20);
    EXPECT_FLOAT_EQ(avl.getMedian(), 20.0f);
}

// 2. Median After Even Inserts
TEST(MedianAVLTest, SimpleEvenMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(20);
    avl.insertValue(30);
```

```

    avl.insertValue(40);
    EXPECT_FLOAT_EQ(avl.getMedian(), 25.0f); // avg(20,30)
}

// 3. Auto-Balancing After Inserts
TEST(MedianAVLTest, BalancingAfterInsert) {
    MedianAVL avl;
    avl.insertValue(50);
    avl.insertValue(30);
    avl.insertValue(40); // triggers LR rotation
    EXPECT_TRUE(avl.isBalanced());
}

// 4. Delete Value Updates Median
TEST(MedianAVLTest, DeleteUpdatesMedian) {
    MedianAVL avl;
    avl.insertValue(10);
    avl.insertValue(20);
    avl.insertValue(30);
    avl.insertValue(40);
    avl.insertValue(50);
    EXPECT_FLOAT_EQ(avl.getMedian(), 30.0f);

    avl.deleteValue(30);
    EXPECT_FLOAT_EQ(avl.getMedian(), 35.0f); // avg(30,40) after deletion
}

// 5. Handle Duplicate Insertion Gracefully
TEST(MedianAVLTest, RejectDuplicates) {
    MedianAVL avl;
    EXPECT_TRUE(avl.insertValue(25.0f));
    EXPECT_FALSE(avl.insertValue(25.0f));
    EXPECT_EQ(avl.getCount(), 1);
}

// 6. Delete Nonexistent Value
TEST(MedianAVLTest, DeleteNonexistentValue) {
    MedianAVL avl;
    avl.insertValue(10);
    EXPECT_FALSE(avl.deleteValue(999)); // no node
}

```

```

    EXPECT_TRUE(avl.isBalanced());
}

// 7. Median After Many Inserts
TEST(MedianAVLTest, MedianAfterBulkInserts) {
    MedianAVL avl;
    float vals[] = {10, 20, 30, 40, 50, 60, 70};
    for (int i = 0; i < 7; ++i)
        avl.insertValue(vals[i]);
    EXPECT_FLOAT_EQ(avl.getMedian(), 40.0f);
}

// 8. Median After Bulk Inserts and Deletions
TEST(MedianAVLTest, BulkInsertAndDelete) {
    MedianAVL avl;
    for (int i = 1; i <= 10; ++i)
        avl.insertValue((float)i);
    EXPECT_FLOAT_EQ(avl.getMedian(), 5.5f);
    for (int i = 1; i <= 5; ++i)
        avl.deleteValue((float)i);
    EXPECT_FLOAT_EQ(avl.getMedian(), 8.0f);
}

// 9. Check Subtree Size Integrity
TEST(MedianAVLTest, SubtreeSizesValid) {
    MedianAVL avl;
    for (int i = 1; i <= 5; ++i)
        avl.insertValue((float)i * 10);
    EXPECT_EQ(avl.getCount(), 5);
    EXPECT_TRUE(avl.isBalanced());
}

// 10. Empty Tree Edge Case
TEST(MedianAVLTest, EmptyTreeMedian) {
    MedianAVL avl;
    EXPECT_EQ(avl.getCount(), 0);
    EXPECT_FLOAT_EQ(avl.getMedian(), 0.0f); // convention: 0 for empty
}

```